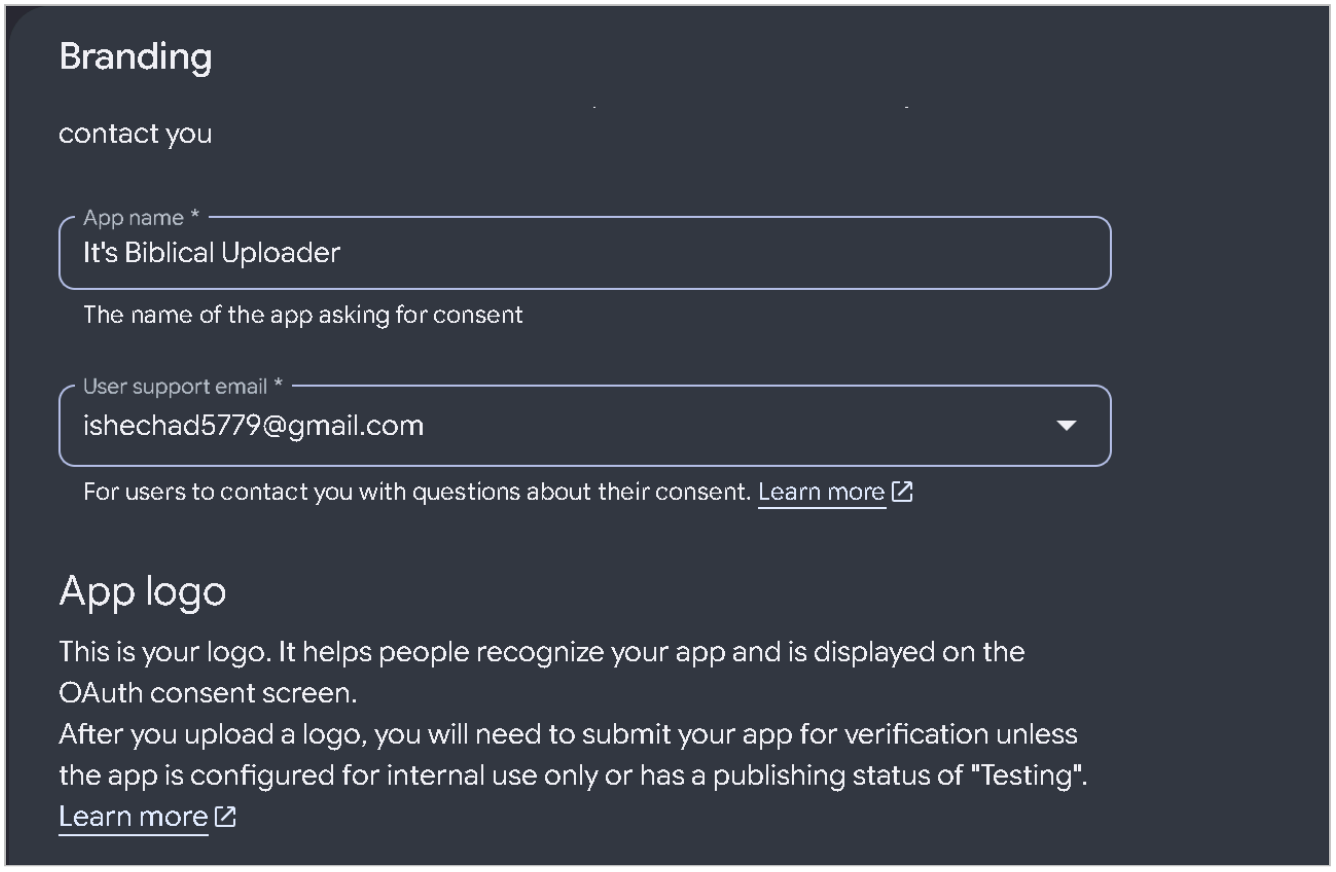
It's Biblical Uploader: API audit evidence (Project 252511401771)

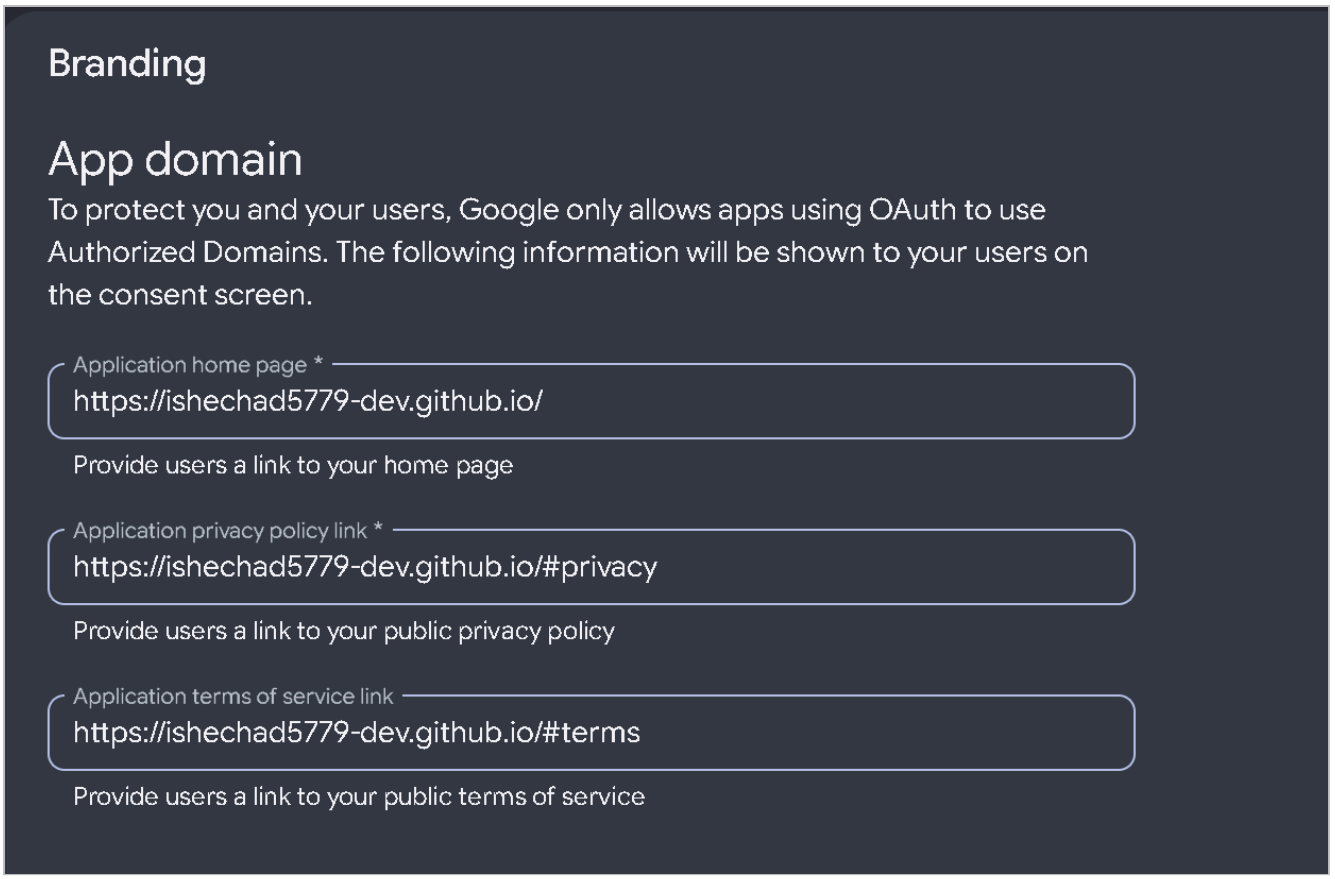
Owner: Eli Eliach (ishechad5779@gmail.com). Channel: https://www.youtube.com/@its.biblical. Website, privacy policy and terms: https://ishechad5779-dev.github.io/

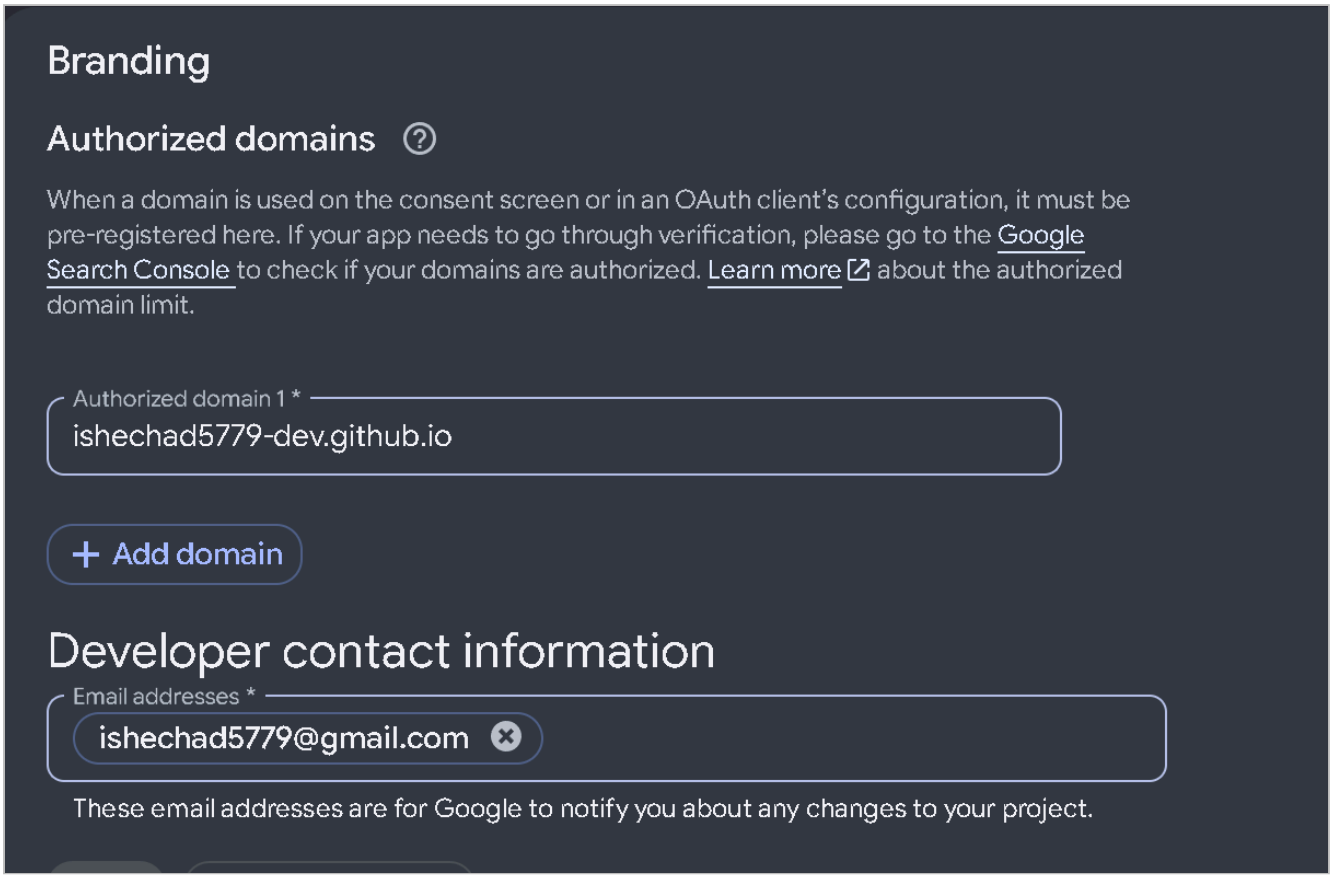
Contents: (a) OAuth flow: consent screen branding, scopes, revocation. (b) Upload tool: command-line interface and the complete, working upload script (also at https://github.com/ishechad5779-dev/ishechad5779-dev.github.io/blob/main/tool/yt_upload.py).

The client is a private command-line tool used only by the channel owner. It authenticates with OAuth 2.0 (scopes youtube.upload and youtube.force-ssl), uploads the owner's own videos with resumable uploads, and sets metadata, thumbnails, captions, playlists and scheduled publishing. Credentials are read from environment variables and are never stored in code.

(a1) OAuth consent screen: branding, app domain, authorized domain

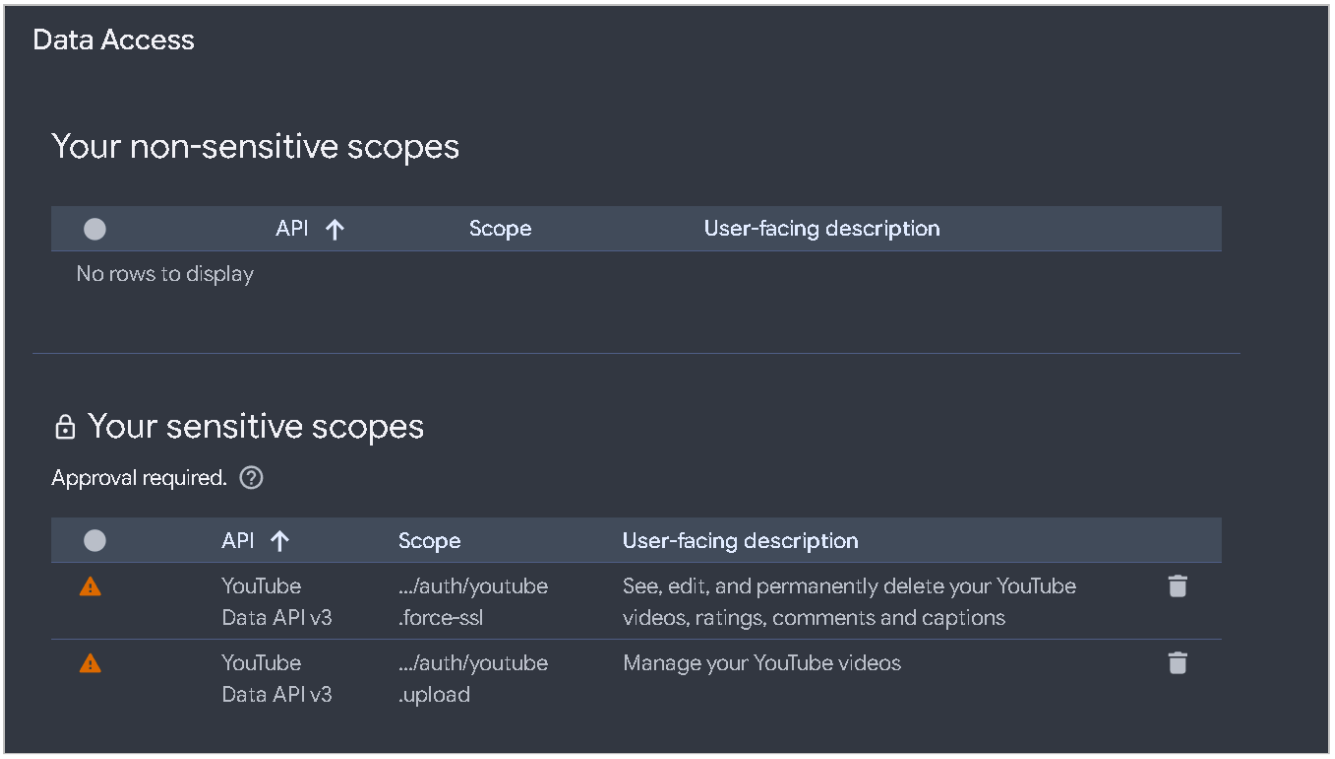


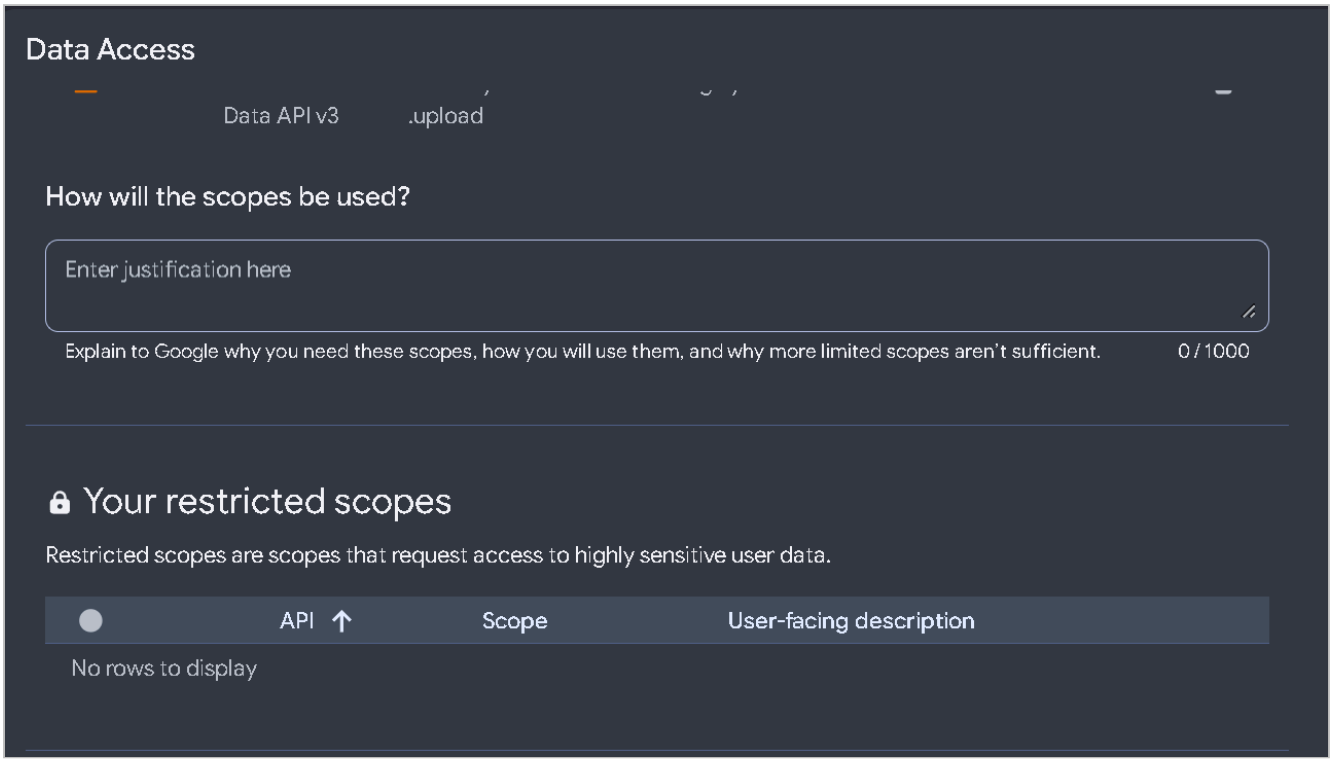

Branding
App domain
To protect you and your users, Google only allows apps using OAuth to use Authorized Domains. The following information will be shown to your users on the consent screen.
Application home page *
https://ishechad5779-dev.github.io/
Provide users a link to your home page
Application privacy policy link *
https://ishechad5779-dev.github.io/#privacy
Provide users a link to your public privacy policy
Application terms of service link
https://ishechad5779-dev.github.io/#terms
Provide users a link to your public terms of service


Branding
Authorized domains
When a domain is used on the consent screen or in an OAuth client's configuration, it must be pre-registered here. If your app needs to go through verification, please go to the Google Search Console to check if your domains are authorized. Learn more about the authorized domain limit.
Authorized domain 1 *
ishechad5779-dev.github.io
+ Add domain
Developer contact information
Email addresses *
ishechad5779@gmail.com
These email addresses are for Google to notify you about any changes to your project.

(a2) OAuth scopes requested

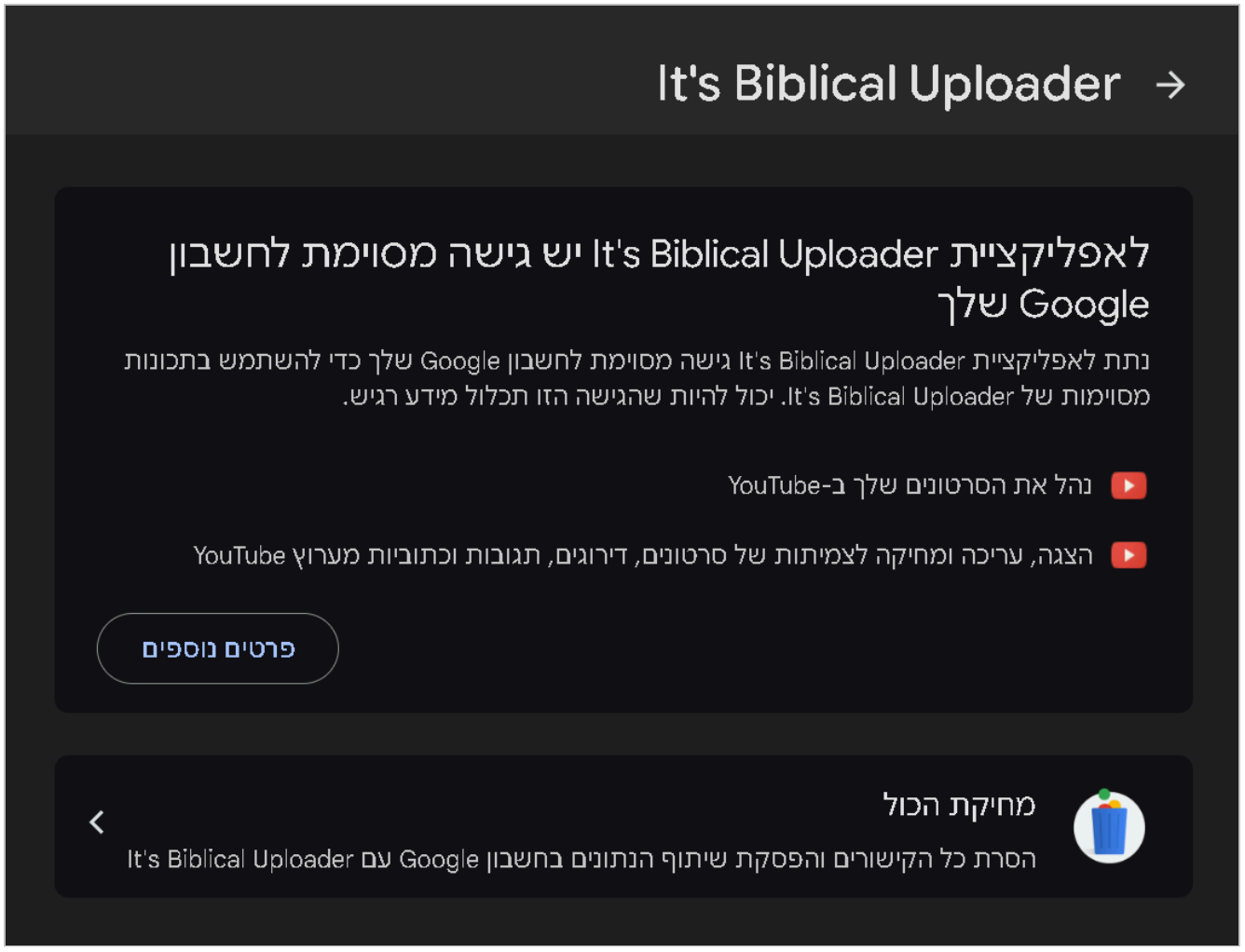
Only two scopes: youtube.upload (upload videos) and youtube.force-ssl (manage metadata, captions and comments on the owner's own channel).





(a3) Revocation: Google Account, Third-party connections

The owner can remove the app's access at any time with the "Delete all connections" button (shown in Hebrew UI).



(b1) Upload tool: command-line interface

Typical run: yt_upload.py upload video.mp4 --title ... --description-file ... --thumbnail thumb.jpg --caption en=en.srt ... --playlist PL... --publish-at 2026-11-04T15:00:00Z --synthetic

```
$ python3 yt_upload.py --help
usage: yt_upload.py [-h]
                    {upload,status,set-status,thumbnail,caption,delete} ...

Direct YouTube Data API v3 uploader for @its.biblical (no Composio).

Credentials come from env vars YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN
(Eli's own Google Cloud project "its-biblical-uploader"). Values are never printed.
Stdlib only.

Quota (10,000 units/day, resets midnight Pacific):
  videos.insert 1600 | thumbnails.set 50 | captions.insert 400 | videos.update 50
  playlistItems.insert 50 | videos.list 1 | videos.delete 50

Until the API audit passes, Google forces every video uploaded through this
project to stay PRIVATE (publishAt will not take effect).

Usage:
  yt_upload.py upload VIDEO.mp4 --title T [--description-file F | --description D]
        [--tags a,b,c] [--category 27] [--privacy private|unlisted|public]
        [--publish-at 2026-11-04T15:00:00Z] [--synthetic | --no-synthetic]
        [--thumbnail thumb.jpg] [--caption en=captions-en.srt --caption he=he.srt ...]
        [--playlist PLxxxx] [--lang en]
  yt_upload.py status VIDEO_ID           # privacy, publishAt, processing state
  yt_upload.py set-status VIDEO_ID [--privacy ..] [--publish-at ..] [--synthetic]
  yt_upload.py thumbnail VIDEO_ID thumb.jpg
  yt_upload.py caption VIDEO_ID LANG FILE.srt [--name NAME]
  yt_upload.py delete VIDEO_ID --confirm-title "exact title"

positional arguments:
  {upload,status,set-status,thumbnail,caption,delete}

options:
  -h, --help            show this help message and exit
```

(b2) Complete upload script: yt_upload.py

```
#!/usr/bin/env python3
"""Direct YouTube Data API v3 uploader for @its.biblical (no Composio).

Credentials come from env vars YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN
(Eli's own Google Cloud project "its-biblical-uploader"). Values are never printed.
Stdlib only.

Quota (10,000 units/day, resets midnight Pacific):
  videos.insert 1600 | thumbnails.set 50 | captions.insert 400 | videos.update 50
  playlistItems.insert 50 | videos.list 1 | videos.delete 50

Until the API audit passes, Google forces every video uploaded through this
project to stay PRIVATE (publishAt will not take effect).

Usage:
  yt_upload.py upload VIDEO.mp4 --title T [--description-file F | --description D]
        [--tags a,b,c] [--category 27] [--privacy private|unlisted|public]
        [--publish-at 2026-11-04T15:00:00Z] [--synthetic | --no-synthetic]
        [--thumbnail thumb.jpg] [--caption en=captions-en.srt --caption he=he.srt ...]
        [--playlist PLxxxx] [--lang en]
  yt_upload.py status VIDEO_ID           # privacy, publishAt, processing state
  yt_upload.py set-status VIDEO_ID [--privacy ..] [--publish-at ..] [--synthetic]
  yt_upload.py thumbnail VIDEO_ID thumb.jpg
  yt_upload.py caption VIDEO_ID LANG FILE.srt [--name NAME]
  yt_upload.py delete VIDEO_ID --confirm-title "exact title"
"""
import argparse, json, mimetypes, os, sys, time, urllib.error, urllib.parse, urllib.request, uuid

API = "https://www.googleapis.com/youtube/v3"
UPLOAD = "https://www.googleapis.com/upload/youtube/v3"
CHUNK = 16 * 1024 * 1024  # multiple of 256 KiB

LANG_NAMES = {"en": "English", "he": "Hebrew", "ar": "Arabic", "es": "Spanish",
              "pt": "Portuguese", "fr": "French", "de": "German", "ru": "Russian"}

_token = {"value": None, "exp": 0}


def token():
    if _token["value"] and time.time() < _token["exp"] - 60:
        return _token["value"]
    missing = [k for k in ("YT_CLIENT_ID", "YT_CLIENT_SECRET", "YT_REFRESH_TOKEN") if not os.environ.get(k)]
    if missing:
        sys.exit(f"missing env vars: {', '.join(missing)}")
    data = urllib.parse.urlencode({
        "client_id": os.environ["YT_CLIENT_ID"],
        "client_secret": os.environ["YT_CLIENT_SECRET"],
        "refresh_token": os.environ["YT_REFRESH_TOKEN"],
        "grant_type": "refresh_token"}).encode()
    try:
        r = json.load(urllib.request.urlopen("https://oauth2.googleapis.com/token", data, timeout=30))
    except urllib.error.HTTPError as e:
        body = e.read().decode()[:300]
        if "invalid_grant" in body:
            sys.exit("refresh token rejected (invalid_grant): expired or revoked. If the OAuth app is in "
                     "Testing mode tokens die after 7 days; publish the app and mint a new token.")
        sys.exit(f"token exchange failed {e.code}: {body}")
    _token.update(value=r["access_token"], exp=time.time() + r.get("expires_in", 3600))
    return _token["value"]


def req(method, url, body=None, headers=None, raw=False, ok=(200, 201, 204)):
    h = {"Authorization": f"Bearer {token()}"}
    if headers:
        h.update(headers)
    if isinstance(body, (dict, list)):
        body = json.dumps(body).encode()
        h.setdefault("Content-Type", "application/json; charset=UTF-8")
    for attempt in range(5):
        r = urllib.request.Request(url, data=body, method=method, headers=h)
        try:
            resp = urllib.request.urlopen(r, timeout=300)
            if raw:
                return resp
            txt = resp.read().decode()
            return json.loads(txt) if txt else {}
        except urllib.error.HTTPError as e:
            if e.code in (500, 502, 503, 504) and attempt < 4:
                time.sleep(2 ** attempt * 2)
                continue
            msg = e.read().decode()[:800]
            raise SystemExit(f"{method} {url.split('?')[0]} -> HTTP {e.code}: {msg}")
        except (urllib.error.URLError, TimeoutError, ConnectionError) as e:
            if attempt < 4:
                time.sleep(2 ** attempt * 2)
                continue
            raise
```

```
def resumable_upload(path, snippet, status):
    size = os.path.getsize(path)
    meta = {"snippet": snippet, "status": status}
    parts = "snippet,status"
    url = f"{UPLOAD}/videos?uploadType=resumable&part={parts}&notifySubscribers=true"
    resp = req("POST", url, meta, headers={
        "X-Upload-Content-Type": "video/*", "X-Upload-Content-Length": str(size)}, raw=True)
    session = resp.headers["Location"]
    sent = 0
    with open(path, "rb") as f:
        while sent < size:
            f.seek(sent)
            chunk = f.read(CHUNK)
            end = sent + len(chunk) - 1
            for attempt in range(6):
                r = urllib.request.Request(session, data=chunk, method="PUT", headers={
                    "Authorization": f"Bearer {token()}", "Content-Length": str(len(chunk)),
                    "Content-Range": f"bytes {sent}-{end}/{size}"})
                try:
                    out = urllib.request.urlopen(r, timeout=600)
                    return json.loads(out.read().decode())  # 200/201: done
                except urllib.error.HTTPError as e:
                    if e.code == 308:  # chunk accepted, more to send
                        rng = e.headers.get("Range")
                        sent = int(rng.split("-")[1]) + 1 if rng else 0
                        break
                    if e.code in (500, 502, 503, 504) and attempt < 5:
                        time.sleep(2 ** attempt * 2)
                        sent = _query_offset(session, size)
                        break
                    raise SystemExit(f"upload chunk failed HTTP {e.code}: {e.read().decode()[:800]}")
                except (urllib.error.URLError, TimeoutError, ConnectionError):
                    if attempt == 5:
                        raise
                    time.sleep(2 ** attempt * 2)
                    sent = _query_offset(session, size)
                    break
            print(f"  uploaded {sent}/{size} bytes", file=sys.stderr)
    raise SystemExit("upload ended without a final response")


def _query_offset(session, size):
    r = urllib.request.Request(session, data=b"", method="PUT", headers={
        "Authorization": f"Bearer {token()}", "Content-Range": f"bytes */{size}"})
    try:
        urllib.request.urlopen(r, timeout=60)
        return size
    except urllib.error.HTTPError as e:
        if e.code == 308:
            rng = e.headers.get("Range")
            return int(rng.split("-")[1]) + 1 if rng else 0
        raise


def multipart(url, meta, path, ctype):
    boundary = uuid.uuid4().hex
    with open(path, "rb") as f:
        data = f.read()
    body = (f"--{boundary}\r\nContent-Type: application/json; charset=UTF-8\r\n\r\n"
            f"{json.dumps(meta)}\r\n--{boundary}\r\nContent-Type: {ctype}\r\n\r\n").encode() \
        + data + f"\r\n--{boundary}--".encode()
    return req("POST", url, body, headers={"Content-Type": f"multipart/related; boundary={boundary}"})


def set_thumbnail(vid, path):
    ctype = mimetypes.guess_type(path)[0] or "image/jpeg"
    with open(path, "rb") as f:
        data = f.read()
    if len(data) > 2 * 1024 * 1024:
        raise SystemExit("thumbnail must be under 2 MB")
    return req("POST", f"{UPLOAD}/thumbnails/set?videoId={vid}&uploadType=media", data,
               headers={"Content-Type": ctype})


def add_caption(vid, lang, path, name=None):
    meta = {"snippet": {"videoId": vid, "language": lang, "name": name or LANG_NAMES.get(lang, lang),
                        "isDraft": False}}
    return multipart(f"{UPLOAD}/captions?uploadType=multipart&part=snippet", meta, path,
                     "application/octet-stream")


def get_video(vid):
    r = req("GET", f"{API}/videos?part=snippet,status,processingDetails,contentDetails&id={vid}")
    return r["items"][0] if r.get("items") else None


def update_status(vid, privacy=None, publish_at=None, synthetic=None):
    cur = get_video(vid)
    if not cur:
        raise SystemExit(f"video {vid} not found")
    st = cur["status"]
```

```
    # videos.update replaces the whole status part: carry existing fields so they don't reset
    new = {k: st[k] for k in ("privacyStatus", "embeddable", "license", "publicStatsViewable",
                              "selfDeclaredMadeForKids", "containsSyntheticMedia", "publishAt") if k in st}
    new.setdefault("selfDeclaredMadeForKids", False)
    if privacy:
        new["privacyStatus"] = privacy
    if publish_at:
        new["privacyStatus"] = "private"
        new["publishAt"] = publish_at
    if synthetic is not None:
        new["containsSyntheticMedia"] = synthetic
    return req("PUT", f"{API}/videos?part=status", {"id": vid, "status": new})


def add_to_playlist(vid, playlist):
    return req("POST", f"{API}/playlistItems?part=snippet",
               {"snippet": {"playlistId": playlist, "resourceId": {"kind": "youtube#video", "videoId": vid}}})


def cmd_upload(a):
    desc = a.description or ""
    if a.description_file:
        with open(a.description_file, encoding="utf-8") as f:
            desc = f.read()
    snippet = {"title": a.title[:100], "description": desc[:5000], "categoryId": a.category,
               "defaultLanguage": a.lang, "defaultAudioLanguage": a.lang}
    if a.tags:
        snippet["tags"] = [t.strip() for t in a.tags.split(",") if t.strip()]
    status = {"privacyStatus": "private" if a.publish_at else a.privacy,
              "selfDeclaredMadeForKids": False, "embeddable": True, "license": "youtube",
              "publicStatsViewable": True}
    if a.publish_at:
        status["publishAt"] = a.publish_at
    if a.synthetic is not None:
        status["containsSyntheticMedia"] = a.synthetic
    print(f"uploading {a.video} ({os.path.getsize(a.video) / 1e6:.1f} MB)", file=sys.stderr)
    v = resumable_upload(a.video, snippet, status)
    vid = v["id"]
    out = {"id": vid, "url": f"https://youtu.be/{vid}", "status": v.get("status")}
    if a.thumbnail:
        set_thumbnail(vid, a.thumbnail)
        out["thumbnail"] = "set"
    caps = []
    for c in a.caption or []:
        lang, path = c.split("=", 1)
        caps.append(add_caption(vid, lang, path)["id"] and lang)
    if caps:
        out["captions"] = caps
    if a.playlist:
        add_to_playlist(vid, a.playlist)
        out["playlist"] = a.playlist
    print(json.dumps(out, indent=2))


def main():
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sp = p.add_subparsers(dest="cmd", required=True)

    u = sp.add_parser("upload")
    u.add_argument("video")
    u.add_argument("--title", required=True)
    u.add_argument("--description")
    u.add_argument("--description-file")
    u.add_argument("--tags")
    u.add_argument("--category", default="27")  # Education
    u.add_argument("--lang", default="en")
    u.add_argument("--privacy", default="private", choices=["private", "unlisted", "public"])
    u.add_argument("--publish-at", help="ISO 8601 UTC, e.g. 2026-11-04T15:00:00Z (forces private until then)")
    g = u.add_mutually_exclusive_group()
    g.add_argument("--synthetic", dest="synthetic", action="store_true", default=None,
                   help="declare altered/synthetic content (containsSyntheticMedia=true)")
    g.add_argument("--no-synthetic", dest="synthetic", action="store_false")
    u.add_argument("--thumbnail")
    u.add_argument("--caption", action="append", help="LANG=path.srt, repeatable")
    u.add_argument("--playlist")

    s = sp.add_parser("status"); s.add_argument("video_id")
    ss = sp.add_parser("set-status"); ss.add_argument("video_id")
    ss.add_argument("--privacy", choices=["private", "unlisted", "public"])
    ss.add_argument("--publish-at")
    ss.add_argument("--synthetic", action="store_true", default=None)
    t = sp.add_parser("thumbnail"); t.add_argument("video_id"); t.add_argument("image")
    c = sp.add_parser("caption"); c.add_argument("video_id"); c.add_argument("lang"); c.add_argument("file")
    c.add_argument("--name")
    d = sp.add_parser("delete"); d.add_argument("video_id")
    d.add_argument("--confirm-title", required=True, help="must equal the video's current title")

    a = p.parse_args()
    if a.cmd == "upload":
        cmd_upload(a)
    elif a.cmd == "status":
        v = get_video(a.video_id)
        if not v:
```

```
            sys.exit("not found")
        print(json.dumps({"id": v["id"], "title": v["snippet"]["title"],
                          "channel": v["snippet"]["channelTitle"], "status": v["status"],
                          "processing": v.get("processingDetails"),
                          "duration": v.get("contentDetails", {}).get("duration")}, indent=2))
    elif a.cmd == "set-status":
        print(json.dumps(update_status(a.video_id, a.privacy, a.publish_at, a.synthetic)["status"], indent=2))
    elif a.cmd == "thumbnail":
        set_thumbnail(a.video_id, a.image); print("thumbnail set")
    elif a.cmd == "caption":
        print(json.dumps(add_caption(a.video_id, a.lang, a.file, a.name)["snippet"], indent=2))
    elif a.cmd == "delete":
        v = get_video(a.video_id)
        if not v:
            sys.exit("not found")
        if v["snippet"]["title"] != a.confirm_title:
            sys.exit(f"refusing: title is {v['snippet']['title']!r}, not {a.confirm_title!r}")
        req("DELETE", f"{API}/videos?id={a.video_id}")
        print(f"deleted {a.video_id}")


if __name__ == "__main__":
    main()
```